

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение

высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/

МЕЖДИСЦИПЛИНАРНЫЙ КУРС

МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО

МАТЕРИАЛА ***Лекция – 4-5***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Лекция 4-5

Жизненный цикл ПО

Модели ЖЦ

Стандарты жизненного цикла ПО

Информационные системы должны удовлетворять интересам бизнеса, а также быть легко модифицируемыми и недорогими. Плохо спроектированная система, в конечном счете, требует больших затрат и времени для ее содержания и обновления.

Одним из базовых понятий методологии проектирования ИС является понятие жизненного цикла ее программного обеспечения (ЖЦ ПО).

Жизненный цикл ПО – это непрерывный процесс, который начинается с момента принятия решения о необходимости его создания и заканчивается в момент его полного изъятия из эксплуатации.

Существует целый ряд стандартов, регламентирующих ЖЦ ПО, а в некоторых случаях и процессы разработки.

Среди наиболее известных стандартов можно выделить следующие:

- ***ГОСТ 34.601-90*** - распространяется на автоматизированные системы и устанавливает стадии и этапы их создания. Кроме того, в стандарте содержится описание содержания работ на каждом этапе. Стадии и этапы работы, закрепленные в стандарте, в большей степени соответствуют каскадной модели жизненного цикла.

- ***ISO/IEC 12207:1995*** - стандарт на процессы и организацию жизненного цикла. Распространяется на все виды заказного ПО. Стандарт не содержит описания фаз, стадий и этапов.

- ***Custom Development Method*** (методика Oracle) по разработке прикладных информационных систем - технологический материал, детализированный до уровня заготовок проектных документов, рассчитанных на использование в проектах с применением Oracle. Применяется CDM для классической модели ЖЦ (предусмотрены все работы/задачи и этапы), а также для технологий "быстрой разработки" (Fast Track) или "облегченного подхода", рекомендуемых в случае малых проектов.

- ***Rational Unified Process (RUP)*** предлагает итеративную модель разработки, включающую четыре фазы: начало, исследование, построение и внедрение. Каждая фаза может быть разбита на этапы (итерации), в результате которых выпускается версия для внутреннего или внешнего использования. Прохождение через четыре основные фазы называется циклом разработки, каждый цикл завершается генерацией версии системы. Если после этого работа над проектом не прекращается, то полученный продукт продолжает развиваться и снова минует те же фазы. Суть работы в рамках RUP - это создание и сопровождение моделей на базе UML.

- ***Microsoft Solution Framework (MSF)*** сходна с RUP, так же включает четыре фазы: анализ, проектирование, разработка, стабилизация, является итерационной, предполагает использование объектно-ориентированного моделирования. MSF в сравнении с RUP в большей степени ориентирована на разработку бизнес-приложений.

- ***Extreme Programming (XP).*** Экстремальное программирование (самая новая среди рассматриваемых методологий) сформировалось в 1996 году. В основе

методологии командная работа, эффективная коммуникация между заказчиком и исполнителем в течение всего проекта по разработке ИС, а разработка ведется с использованием последовательно дорабатываемых прототипов.

Основные процессы жизненного цикла ISO/IEC 12207

В соответствии с базовым международным стандартом ISO/IEC 12207 все процессы ЖЦ ПО делятся на три группы:

1. **Основные процессы:**
   - приобретение;
   - поставка;
   - разработка;
   - эксплуатация;
   - сопровождение.
2. **Вспомогательные процессы:**
   - документирование;
   - управление конфигурацией;
   - обеспечение качества;
   - разрешение проблем;
   - аудит;
   - аттестация;
   - совместная оценка;
   - верификация.
3. **Организационные процессы:**
   - создание инфраструктуры;
   - управление;
   - обучение;
   - усовершенствование.

В таблице 1 приведены ориентировочные описания основных процессов ЖЦ. Вспомогательные процессы предназначены для поддержки выполнения основных процессов, обеспечения качества проекта, организации верификации, проверки и тестирования ПО. Организационные процессы определяют действия и задачи, выполняемые как заказчиком, так и разработчиком проекта для управления своими процессами.

Для поддержки практического применения стандарта ISO/IEC 12207 разработан ряд технологических документов:

- Руководство для ISO/IEC 12207 (ISO/IEC TR 15271:1998 Information technology - Guide for ISO/IEC 12207)
- Руководство по применению ISO/IEC 12207 к управлению проектами (ISO/IEC TR 16326:1999 Software engineering - Guide for the application of ISO/IEC 12207 to project management).

Таблица 1. Содержание основных процессов ЖЦ ПО ИС (ISO/IEC 12207)

Процесс (исполнитель процесса)	**Действия**	**Вход**	**Результат**
Приобретение (заказчик)	• Инициирование • Подготовка заявочных предложений • Подготовка договора • Контроль деятельности поставщика • Приемка ИС	• Решение о начале работ по внедрению ИС • Результаты обследования деятельности заказчика • Результаты анализа рынка ИС/ тендера • План поставки/ разработки • Комплексный тест ИС	• Технико-экономическое обоснование внедрения ИС • Техническое задание на ИС • Договор на поставку/ разработку • Акты приемки этапов работы • Акт приемно-сдаточных испытаний
Поставка (разработчик ИС)	• Инициирование • Ответ на заявочные предложения • Подготовка договора • Планирование исполнения • Поставка ИС	• Техническое задание на ИС • Решение руководства об участии в разработке • Результаты тендера • Техническое задание на ИС • План управления проектом • Разработанная ИС и документация	• Решение об участии в разработке • Коммерческие предложения/ конкурсная заявка • Договор на поставку/ разработку • План управления проектом • Реализация/ корректировка • Акт приемно-сдаточных испытаний

Разработка (разработчик ИС)	• Подготовка • Анализ требований к ИС • Проектирование архитектуры ИС • Разработка требований к ПО • Проектирование архитектуры ПО • Детальное проектирование ПО • Кодирование и тестирование ПО • Интеграция ПО и квалификационное тестирование ПО • Интеграция ИС и квалификационное тестирование ИС	• Техническое задание на ИС • Техническое задание на ИС, модель ЖЦ • Подсистемы ИС • Спецификации требования к компонентам ПО • Архитектура ПО • Материалы детального проектирования ПО • План интеграции ПО, тесты • Архитектура ИС, ПО, документация на ИС, тесты	• Используемая модель ЖЦ, стандарты разработки • План работ • Состав подсистем, компоненты оборудования • Спецификации требования к компонентам ПО • Состав компонентов ПО, интерфейсы с БД, план интеграции ПО • Проект БД, спецификации интерфейсов между компонентами ПО, требования к тестам • Тексты модулей ПО, акты автономного тестирования • Оценка соответствия комплекса ПО требованиям ТЗ • Оценка соответствия ПО, БД, технического комплекса и комплекта документации требованиям ТЗ

ISO/IEC 12207 (ISO – International Organization of Standardization – Международная организация по стандартизации; IEC – International Electrotechnical Commission – международная комиссия по электротехнике) определяет структуру ЖЦ, содержащую процессы, действия и задачи, которые должны быть выполнены во время создания ПО.

Эксплуатация включает в себя работы по внедрению компонентов ПО в эксплуатацию, в том числе конфигурирование базы данных и рабочих мест пользователей, обеспечение эксплуатационной документацией, проведение обучения персонала и т.д., и непосредственно эксплуатацию, в том числе локализацию проблем и устранение причин их возникновения.

Управление проектом связано с вопросами планирования и организации работ, создания коллективов разработчиков и контроля за сроками и качеством выполняемых работ. Техническое и организационное обеспечение проекта включает выбор методов и инструментальных средств для реализации проекта, определение методов описания промежуточных состояний разработки, разработку методов и средств испытаний ПО.

Верификация – это процесс определения того, отвечает ли текущее состояние разработки, достигнутое на данном этапе, требованиям этого этапа. Проверка позволяет оценить соответствие параметров разработки исходным требованиям. Проверка частично совпадает с тестированием, которое связано с идентификацией различий между действительными и ожидаемыми результатами и оценкой соответствия характеристик ПО исходным требованиям.

Управление конфигурацией является одним из вспомогательных процессов, поддерживающих основные процессы жизненного цикла ПО, прежде всего процессы разработки и сопровождения ПО. При создании проектов сложных ИС, состоящих из многих компонентов, каждый из которых может иметь разновидности или версии, возникают проблемы учета их связей и функций, создания унифицированной структуры и обеспечения развития всей системы. Управление конфигурацией позволяет организовать внесение изменений в ПО на всех стадиях ЖЦ. Общие принципы и рекомендации конфигурационного учета, планирования и управления конфигурациями ПО отражены в проекте стандарта ***ISO 12207-2***.

Каждый процесс характеризуется определенными задачами и методами их решения, исходными данными, полученными на предыдущем этапе, и результатами. Результатами анализа, в частности, являются функциональные модели, информационные модели и соответствующие им диаграммы. ЖЦ ПО носит итерационный характер: результаты очередного этапа часто вызывают изменения в проектных решениях, выработанных на более ранних этапах.

Модели жизненного цикла ПО

Стандарт ISO/IEC 12207 не предлагает конкретную модель ЖЦ и методы разработки ПО (под моделью ЖЦ понимается структура, определяющая последовательность выполнения и взаимосвязи процессов, действий и задач, выполняемых на протяжении ЖЦ. Модель ЖЦ зависит от специфики ИС и условий, в которых последняя создается и функционирует). Его регламенты являются общими для любых моделей ЖЦ, методологий и технологий разработки. Стандарт ISO/IEC 12207 описывает структуру процессов ЖЦ ПО, но не конкретизирует в деталях, как реализовать или выполнить действия и задачи, включенные в эти процессы.

В изначально существовавших однородных ИС каждое приложение представляло собой единое целое. Для разработки такого типа приложений применялся каскадный способ. Его основной характеристикой является разбиение всей разработки на этапы, причем переход с одного этапа на следующий происходит только после того, как будет полностью завершена работа на текущем (рис. 2.1). Каждый этап завершается выпуском полного комплекта документации, достаточной для того, чтобы разработка могла быть продолжена другой командой разработчиков.

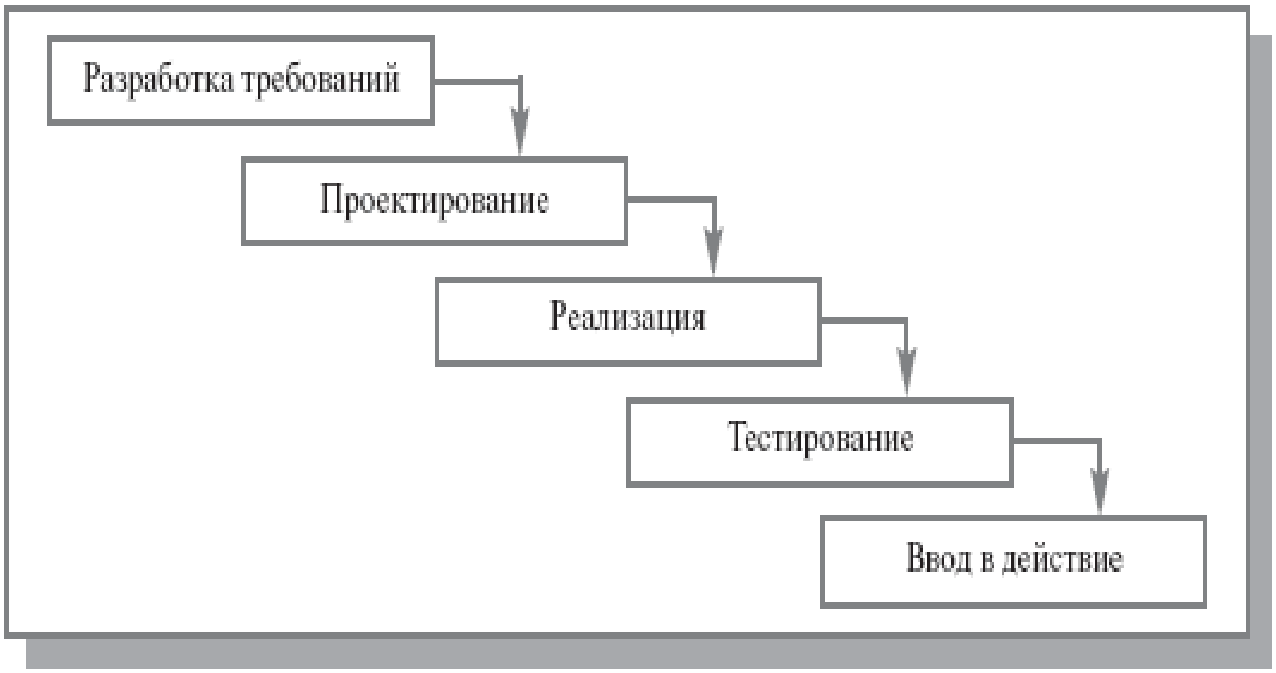


Рис. 2.1 Каскадная схема разработки ПО

Положительные стороны применения каскадного подхода заключаются в следующем:

- на каждом этапе формируется законченный набор проектной документации, отвечающий критериям полноты и согласованности;
- выполняемые в логичной последовательности этапы работ позволяют планировать сроки завершения всех работ и соответствующие затраты.

Каскадный подход хорошо зарекомендовал себя при построении ИС, для которых в самом начале разработки можно достаточно точно и полно сформулировать все

требования, с тем, чтобы предоставить разработчикам свободу реализовать их как можно лучше с технической точки зрения. В эту категорию попадают сложные расчетные системы, системы реального времени и другие подобные задачи. Однако в процессе использования этого подхода обнаружился ряд его недостатков, вызванных прежде всего тем, что реальный процесс создания ПО никогда полностью не укладывался в такую жесткую схему. В процессе создания ПО постоянно возникала потребность в возврате к предыдущим этапам и уточнении или пересмотре ранее принятых решений. В результате реальный процесс создания ПО принимал вид, представленный на рис. 2.2.

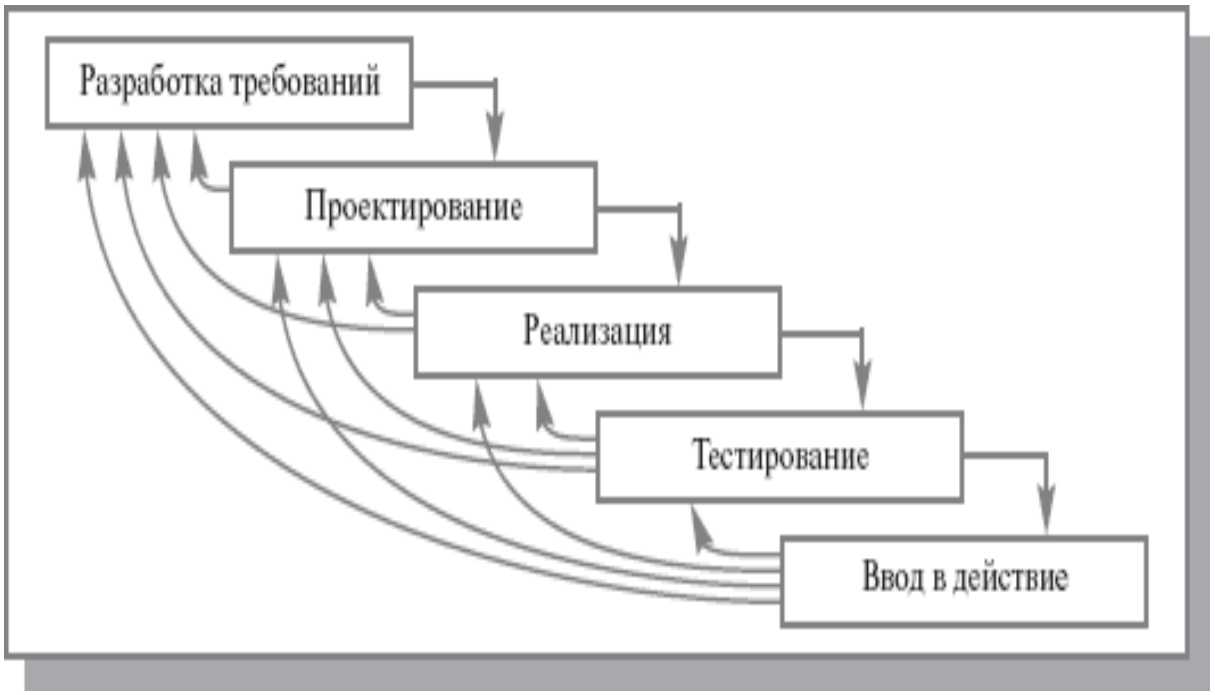
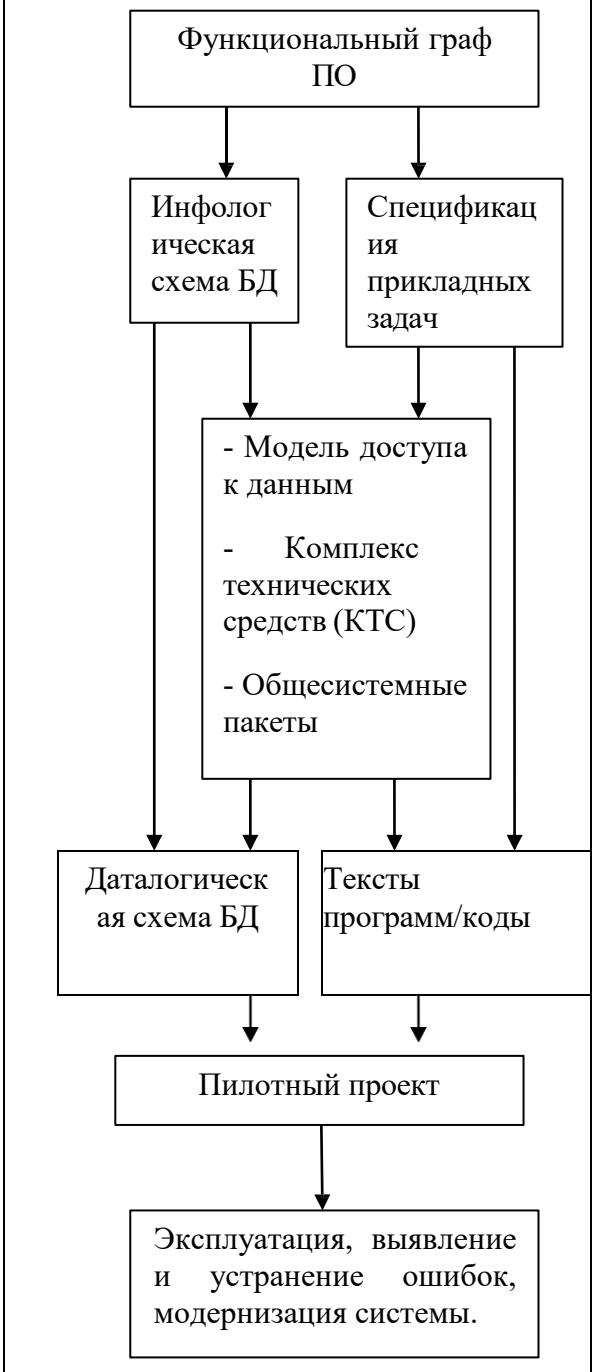


Рис. 2.2 Реальный процесс разработки ПО по каскадной схеме

Рассмотрим более подробно каскадную модель:

Этап	**Результат**
1. Выявление информационных потребностей конечных пользователей (предпроектное обследование, разработка ТЗ, частных ТЗ) 2. Концептуальный проект (эскизное проектирование) 3. Архитектура системы (технический проект) 4. Логическое проектирование (техническое проектирование) 5. Комплексная отладка (разработка рабочей документации) 6. Сопровождение.	

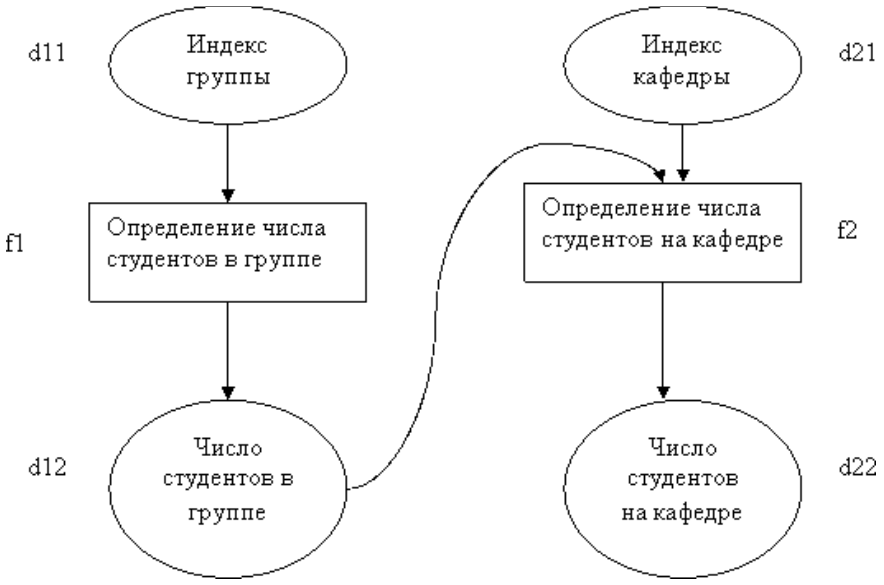
Особенность модели: каждый следующий этап проектирования начинается после полного завершения работ по предыдущему этапу.

1. Выявление информационных потребностей конечных пользователей

Функциональный граф ПО - граф, узлы которого обозначают данные и процессы будущей системы. Дуги используются для обозначения входных/выходных данных

для процесса.

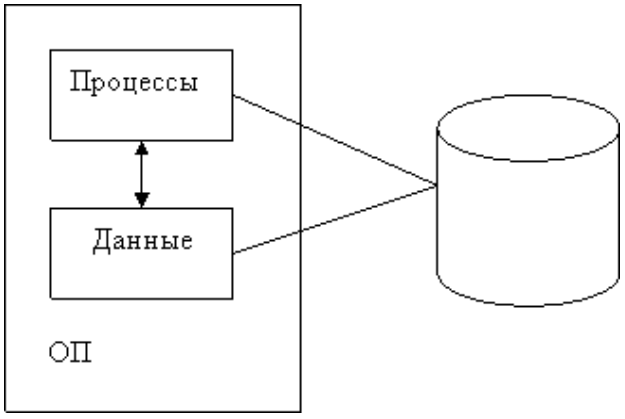
Пример:



d22=f2(d12,d21)=f2(f1(d11),d21)

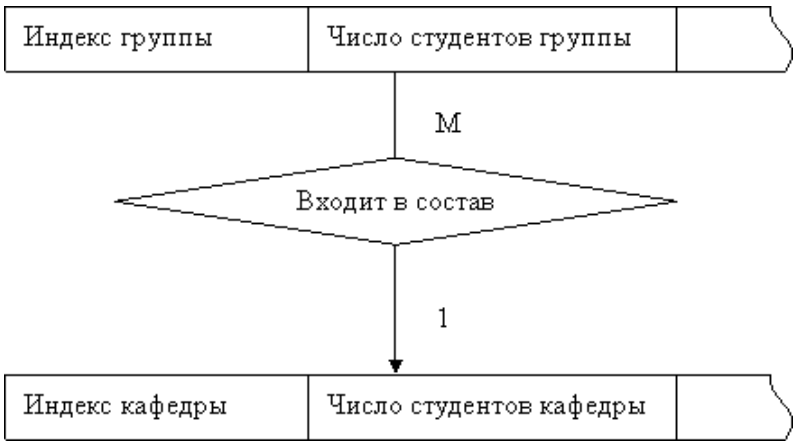
В функциональном графе данные и процессы объединены и в принципе его достаточно для реализации будущей системы (см. формулу к рисунку), но современные компьютеры есть машины Фон-Неймана, где предполагается разделение процессов и данных.

2. Концептуальный проект



Для реализации концептуальной модели проектировщик вынужден выделять из функционального графа данные и строить для них схему БД, а также выделять процессы и разрабатывать для них спецификацию и кодировать. Это является источником большинства ошибок проектирования. Данные функционального графа структурируются в виде инфологической схемы БД.

Пример: (Инфологическая модель БД в нотации Чена)



Спецификация процессов - входные и выходные данные процессов, а также алгоритмическая связь между ними. Для описания спецификации существуют различные методы: структурированный естественный язык (часто используется), язык проектирования спецификации Flow-Form (визуальные языки).

Концептуальный проект не зависит от архитектуры!

3. Выбор архитектуры

- выбор модели доступа к данным (файл-сервер, сервер-БД, сервер-приложение, доступ к данным по Internet/Intarnet)
- выбор комплекса технических средств (выбор «железа»)
- выбор общесистемных пакетов
-выбор способа тиражирования данных

4. Логическое проектирование

Выполняется отражение концептуального проекта в СУБД-ориентированную среду с помощью выбранных оболочек программирования. Сущности преобразуются в таблицы, а на основе спецификации задач разрабатываются тексты программ

Логический проект зависит от архитектуры (можно считать временные характеристики)

5. Отладка

Результаты проектирования БД и приложений объединяются. В итоге разрабатывается пилотный проект системы

6. Сопровождение

Выявление ошибок и их устранение, модернизация.

Достоинства каскадной модели:

- проста, естественна, имеет некоторую привязку к ГОСТу

Недостатки:

- достаточно продолжительный цикл разработки по времени (система морально устаревает)

- доработка системы связана с большим объемом перепрограммирования (из-за слабого использования CASE-средств)

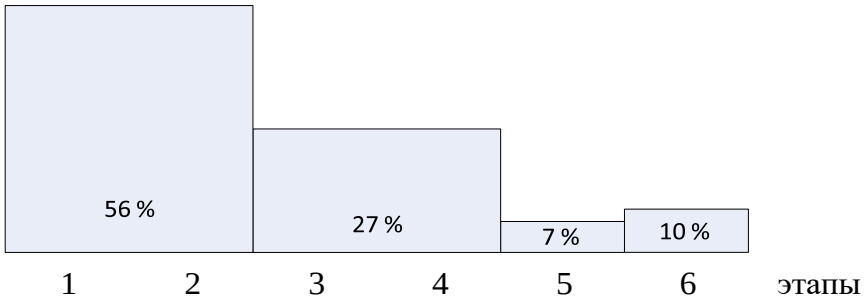
Основным недостатком каскадного подхода является существенное запаздывание с получением результатов. Согласование результатов с пользователями производится только в точках, планируемых после завершения каждого этапа работ, требования к ИС «заморожены» в виде технического задания на все время ее создания.

Таким образом, пользователи могут внести свои замечания только после того, как работа над системой будет полностью завершена. В случае неточного изложения требований или их изменения в течение длительного периода создания ПО пользователи получают систему, не удовлетворяющую их потребностям. Модели (как функциональные, так и информационные) автоматизируемого объекта могут устареть одновременно с их утверждением.

Результаты исследований Д.Мартина (сер. 80х)

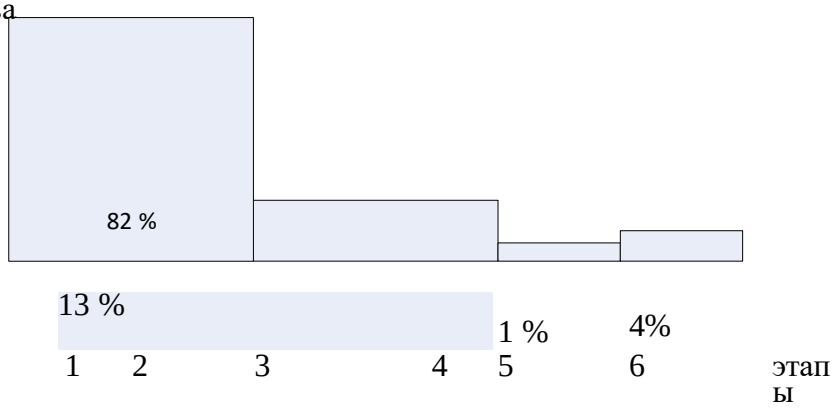
1-я диаграмма: распределение ошибок и просчетов по этапам проектирования, выявленных при сопровождении системы

2-ошибки



3-я диаграмма: распределение затрат на исправление ошибок и просчетов, выявленных при сопровождении

средства



4-я диаграмма: распределение трудозатрат по этапам проектирования

средства



Почти половина трудозатрат приходится на устранение ошибок, допущенных на первых 2-х этапах.

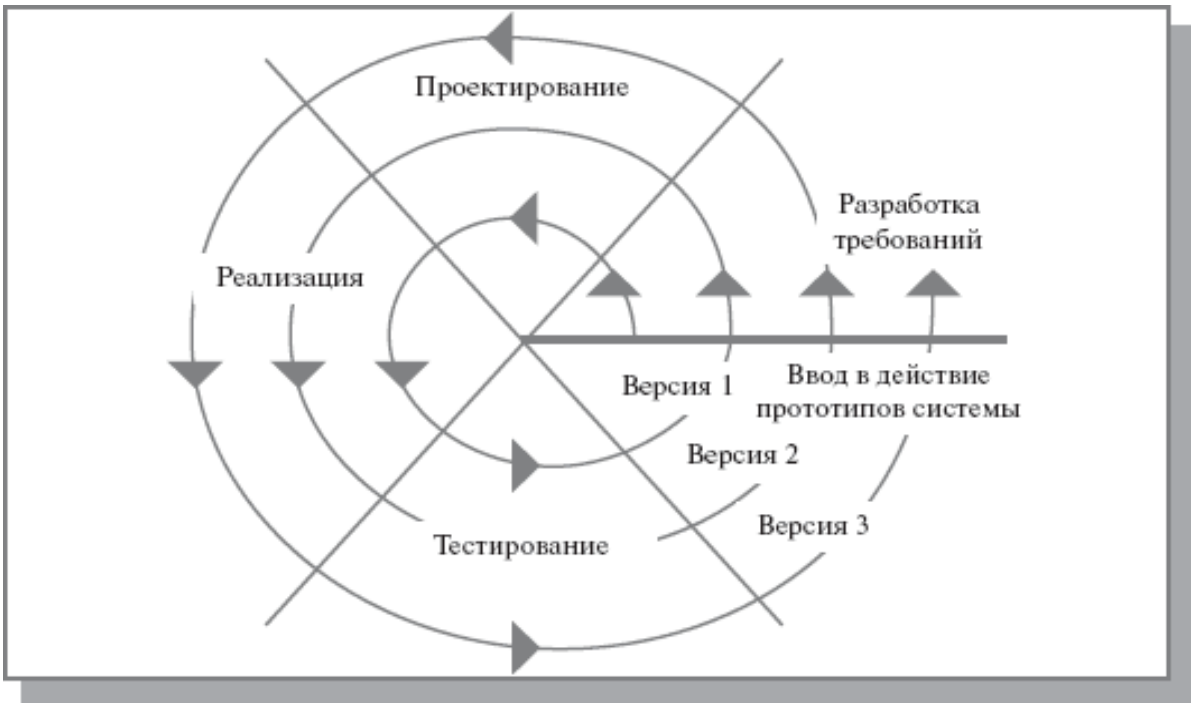
На основании исследований Мартин сформулировал законы:

1. **закон неопределенности в информатике**: процесс автоматизации задачи меняет представление пользователя об этой задаче, т.е. пользователь решает задачу с использованием средств автоматизации иначе, чем без них (пользователя надо использовать постоянно в процессе проектирования, а не только в начале)
2. чем больше времени прошло с момента совершения ошибки до момента ее обнаружения, тем больше средств необходимо для ее устранения (смотри диаграмму 2)
3. программисты и проектировщики не учатся на чужих ошибках, а только на своих.

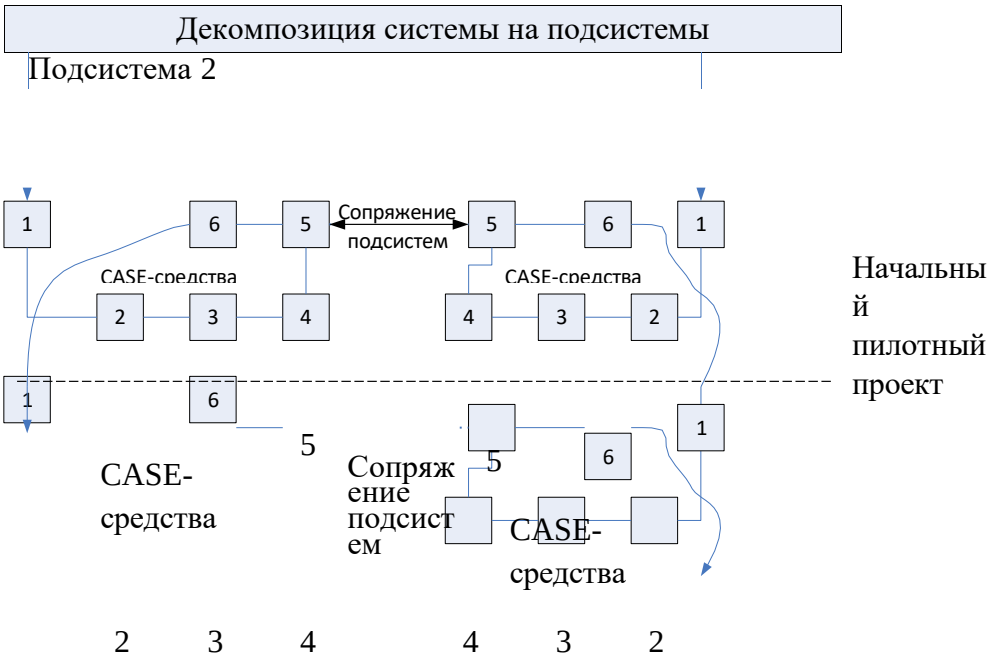
Спиральная модель

Для преодоления перечисленных проблем была предложена спиральная модель ЖЦ делающая упор на начальные этапы ЖЦ: анализ и проектирование. На этих этапах реализуемость технических решений проверяется путем создания прототипов. Каждый виток спирали соответствует созданию фрагмента или версии ПО, на нем уточняются цели и характеристики проекта, определяется его качество и планируются работы следующего витка спирали. Таким образом углубляются и последовательно конкретизируются детали проекта и в результате выбирается обоснованный вариант, который доводится до реализации.

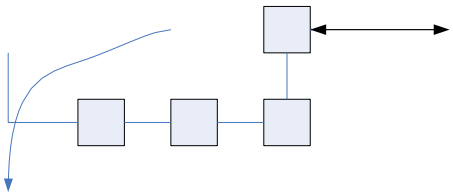
Разработка итерациями отражает объективно существующий спиральный цикл создания системы. Неполное завершение работ на каждом этапе позволяет переходить на следующий этап, не дожидаясь полного завершения работы на текущем. При итеративном способе разработки недостающую работу можно будет выполнить на следующей итерации. Главная же задача – как можно быстрее показать пользователям системы работоспособный продукт, тем самым активизируя процесс уточнения и дополнения требований.



Основная проблема спирального цикла – определение момента перехода на следующий этап. Для ее решения необходимо ввести временные ограничения на каждый из этапов жизненного цикла. Переход осуществляется в соответствии с планом, даже если не вся запланированная работа закончена. План составляется на основе статистических данных, полученных в предыдущих проектах, и личного опыта разработчиков.



1. выбор архитектуры подсистем
2. выявление информационных потребностей конечных пользователей системы
3. концептуальное проектирование
4. логическое проектирование
5. отладка подсистем
6. сопровождение подсистемы



Каждый этап реализуется CASE-средствами. Содержание этапов совпадает с аналогичными в каскадной модели, но в отличие от нее, этапы реализуются с помощью CASE-средств (Computer Aided Software System Design) – **(автоматизированная технология создания программных информационных систем)** – использование этих средств позволяет существенно снизить время реализации витка спирали проектирования

подсистем (в этом состоит основное преимущество спиральной модели), но это профессиональные средства, непредназначенные для конечных пользователей.

С помощью CASE-средств можно быстро сгенерировать проект хотя бы на уровне экранных форм и показать его конечному пользователю, который высказывает свои предложения и замечания, и на следующем витке реализуются они. Когда спецификация на уровне экранных форм будет будет согласована, то проектировщик может начинать детальную реализацию. Описанная схема разработки называют визуальным проектированием.

Витков может быть много, разделение на этапы условно. Работы могут выполняться параллельно или в комплексной итерации на любом витке спирали.

Проектирование как конструирование

Идея конструирования информационных систем (ИС) из компонентов развивается на протяжении многих лет. По существу, речь идет о крупноблочном конструировании систем (Д. Видерхольд). Согласно этой идее, целесообразнее осуществлять капиталовложения в создание компонентов, которые можно было бы многократно использовать, чем всякий раз осуществлять разработку ИС сверху – вниз: от спецификаций требований к работающей системе.

Во-первых, следует отметить то обстоятельство, что компонентно-ориентированный подход к проектированию и реализации программных систем и комплексов является в некотором смысле развитием объектно-ориентированного и практически более пригоден для разработки крупных и распределенных систем (например, корпоративных приложений).

Прежде всего, сформулируем основополагающее для рассматриваемого подхода определение компонента. Под компонентом будем далее иметь в виду независимый модуль программного кода, предназначенный для повторного использования и развертывания. Как видно из определения, применение компонентного программирования призвано обеспечить более простую, быструю и прямолинейную процедуру первоначальной инсталляции прикладного программного обеспечения, а также увеличить процент повторного использования кода, т.е. усилить основные преимущества ООП.

Говоря о свойствах компонентов, следует прежде всего отметить, что это существенно более крупные единицы, чем объекты (в том смысле, что объект представляет собой конструкцию уровня языка программирования). Другими отличиями компонентов от традиционных объектов являются возможность содержать множественные классы и (в большинстве случаев) независимость от языка программирования.

Естественным требованием к современным информационным системам является способность наращивания их возможностей за счет использования дополнительно разработанных (а еще лучше - уже существующих) программных компонентов. Для этого требуется обеспечение свойства, называемого интероперабельностью.

Интероперабельность программного обеспечения (функциональность программного обеспечения) - способность программного продукта выполнять набор функций, определенных в его внешнем описании и удовлетворяющих заданным или

подразумеваемым потребностям пользователей.

Под этим понимается соблюдение определенных правил или привлечение дополнительных программных средств, обеспечивающих возможность взаимодействия независимо от разработанных программных модулей, подсистем или даже функционально завершенных программных систем.